ECHOSCAPE: Land Use / Land Cover segmentation from Sentinel-1 radar alone

Yuvraj Ghag · SRH Hochschule Heidelberg, Deep Learning · 9 November 2026

0. What this is, and what the numbers mean

A semantic segmentation model that maps land cover from **Sentinel-1 radar only**. No optical imagery reaches the model at any point. Alongside it there is a prototype that takes a latitude, longitude and start month and returns twelve monthly land-cover maps with per-class area percentages and month-to-month change.

65.0%
mean IoU, 6 main classes, TEST
temporal_mstat, the best model this project trained

66.7%
overall accuracy, TEST
SegFormer, the model the prototype serves

46.6%
mean IoU, 6 main classes, TEST
the same prototype model

31.1%
mean IoU, all 9 classes, TEST
three of the nine score near zero; §0.1 explains why

Every card above is a **test** figure. Validation scores higher throughout this report, at 51.3% over nine classes and 75.5% over the six main ones, and the gap is the point rather than a problem. Validation is six regions each dominated by a single class, and it is what the architecture, the best epoch and the served model were all selected on. Test is seven mixed regions that took part in no decision at all. A score you selected on is not an estimate of performance on new ground, so the test column is the one that leads.

Two things must be said before any number in this report is read.

0.1 How to read the numbers

Three different measurements appear throughout, and they are not interchangeable.

	WHAT IT MEASURES	SEGFORMER, TEST
Overall accuracy (OA)	of all labelled pixels, the share the model got right	**66.7%**
IoU, per class	for one class: pixels correctly given that class, divided by every pixel that either is that class or was called it. It punishes misses and false alarms alike	water 57.9%, grass 11.7%
mIoU	the plain average of the per-class IoUs, **every class counting equally no matter how rare**	**31.1%** over 9 classes

The gap between 66.7% and 31.1% is not a contradiction, and it is worth being clear about because the two are easy to confuse. Two out of three pixels are labelled correctly. The mean IoU is much lower because it averages over classes rather than pixels, so snow, which is 0.09% of the test pixels and scores zero, pulls the mean down exactly as hard as trees, which is 29.8% of them and scores 65%. **The model is not "31% accurate".** It is right about two thirds of the time, and its class-averaged overlap score is 31.1% because three of its nine classes barely work.

Every score is agreement with Google Dynamic World, not with ground truth. The labels are machine-made from Sentinel-2. Dynamic World is itself 73–84% accurate in published comparisons, so it is a ceiling as well as a reference. Where this report says "accuracy", it means "agreement with a label source that is itself wrong some of the time".

There are two versions of the dataset, and their numbers are not interchangeable. The main results (6 November) use 31 regions, of which 18 are training. On 9 November six regions were added for the three rare classes, giving 37 regions and 24 training regions, and the experiment sweep in notebook 08 ran on that larger set. A model scoring 51.3% on the first and 36.6% on the second has not got worse. It was measured on different ground. Every table below states which dataset it belongs to. Mixing them would be the easiest way to draw a false conclusion from this report, so they are never mixed.

---

1. Dataset and preprocessing

1.1 Source

Radar	**Sentinel-1 RTC**, collection `sentinel-1-rtc`, Microsoft Planetary Computer
Optical (labels only)	Sentinel-2 via **Google Dynamic World**
Region vetting	**ESA WorldCover 2021 (v200)** - used to choose regions, never as a label
Year	**2021**
Resolution	**50 m**
Access	anonymous; no account or API key is needed to run anything except regenerating labels

Why Sentinel-1 RTC and not raw GRD. RTC is Radiometrically Terrain Corrected gamma0: the product is already calibrated and terrain-flattened. This is the honest answer to the "calibration and terrain correction" requirement: those steps are done, but by the data provider, not re-implemented here. It also means the project never had to own a DEM pipeline, which on a 10-week schedule was the difference between finishing and not.

Why 2021. Sentinel-1A and 1B were both operating, giving 6–8 passes per month. Sentinel-1B failed in December 2021, so 2022–2024 have roughly half the coverage. Even monthly coverage matters because the task is month-to-month change: a month with two passes is noisier than a month with eight, and that noise would read as change.

Why 50 m and not 10 m. The idea was to train at a middle, general resolution rather than at the finest one available. A model that has only ever seen 10 m detail learns the texture of that scale and has nothing to fall back on when the input is coarser; training at 50 m aims instead at the structure that survives at many scales, so one model can serve both finer and coarser inputs. 25 m was considered and set aside as too close to the native 10 m to be a meaningfully general scale.

A second reason turned out to matter as much. Radar carries heavy speckle, and averaging pixels together removes most of it, so the coarser data is radiometrically *cleaner*, not merely smaller. Each 50 m pixel is the average of 25 native ones.

In fairness to the idea, it was not tested. Nothing in this project measures the model on a resolution other than the one it trained on, and a network is not automatically scale-invariant: feeding 30 m imagery to these weights would make every feature appear about 1.7 times larger than anything the model has seen. Demonstrating the claim would need either a multi-scale training schedule or a like-for-like test at a second resolution, and both are listed in §7.

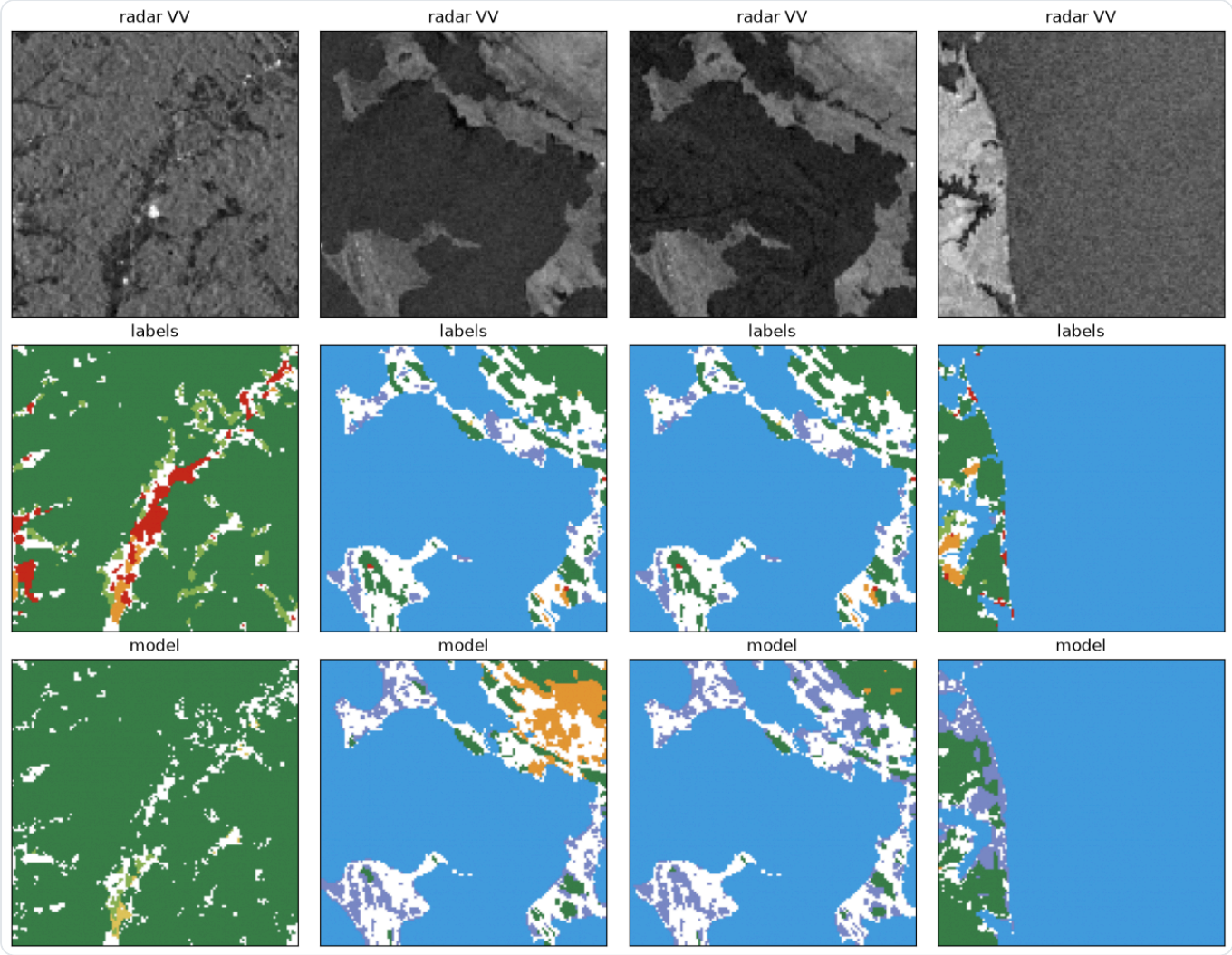


Figure 1. Radar in, land cover out. Top: the VV channel the model reads. Middle: the Dynamic World label. Bottom: the model's prediction from radar alone. Four validation tiles, SegFormer.

1.2 Regions

37 regions worldwide: 24 training, 6 validation, 7 test. Every split is by *whole region*, never by tile.

- **Validation.** One region dominated by each of the six main classes: chesapeake (water), paris (built-up), black_forest (trees), beauce (cropland), serengeti (grass), namib (bare).
- **Test.** lake_geneva, tokyo, congo, kansas, ireland, death_valley, mannheim.
- **Training.** 24 regions, including six added on 9 November specifically for the rare classes: vatnajokull and baltoro (snow & ice), sudd and everglades (flooded vegetation), sonoran and kalahari (shrub & scrub).

Regions were not chosen by eye. Each candidate was checked for Sentinel-1 coverage in all twelve months on a single orbit track, and for its class mix against ESA WorldCover. That check rejected two obvious-sounding picks: a "Kazakh steppe" box that was 81% lake, and a Lake Victoria box with no shoreline in it.

Splitting by whole region is the single most consequential methodological choice in this project. Neighbouring tiles overlap in content even when they do not overlap in pixels, because the same river runs through both. A random tile split would have put near-identical tiles on both sides and inflated every score. It is the reason the numbers in this report are lower than a tile-split project would show, and the reason they mean something.

1.3 Classes and class distribution

All nine Dynamic World classes are kept, none merged: water, built-up, trees, cropland, grass, bare, shrub & scrub, flooded vegetation, snow & ice. Class 0 is *unlabelled* and is excluded from both the loss and every score.

Final dataset (24 training regions), labelled pixels:

CLASS	TRAIN	%	VAL	%	TEST	%
water	8,211,912	8.5	4,168,440	15.7	2,902,647	10.0
built-up	15,467,580	16.0	4,184,028	15.8	6,531,702	22.5
trees	12,604,649	13.1	5,075,052	19.1	8,656,623	29.8
cropland	18,989,892	19.7	4,702,548	17.7	4,718,091	16.3
grass	4,802,976	5.0	3,373,332	12.7	1,246,460	4.3
bare	20,003,904	20.8	4,919,256	18.5	4,830,276	16.6
shrub & scrub	6,604,476	6.9	110,676	**0.42**	84,700	0.29
flooded vegetation	1,967,364	2.0	21,768	**0.08**	16,811	0.06
snow & ice	7,742,726	8.0	**324**	**0.001**	26,076	0.09
total	**96,395,479**		**26,555,424**		**29,013,386**	
tiles	7,164		1,800		2,100	
regions	24		6		7	

Three facts this table makes unavoidable:

1. **Training holds 63% of the labelled pixels.** 37% is spent on measurement. That is generous, and it is a cost paid for honest evaluation.
2. **The effective sample size is 24 ground positions, not 7,164 tiles.** The twelve months of one region are the same ground seen twelve times. This, more than any architectural choice, is what limits the model.
3. **Validation holds 324 snow pixels.** Three of nine classes hold 0.5% of validation between them yet carry one third of the nine-class mean. The six new training regions lifted shrub, flooded vegetation and snow from 1.16%, 0.03% and 0.87% of training pixels to 6.9%, 2.0% and 8.0%. That is a real fix, and the headline metric is structurally unable to register it.

Before the six regions were added, the first dataset's training shares were: water 9.9%, built-up 20.4%, trees 15.3%, cropland 25.0%, grass 5.7%, bare 21.7%, shrub 1.16%, flooded **0.026%**, snow 0.875%. Flooded vegetation amounted to about one tile's worth of pixels in the entire training set.

1.4 Preprocessing, step by step

Calibration and terrain correction. Inherited from the Sentinel-1 RTC product (gamma0, terrain-flattened). Not re-implemented.

Orbit track selection. One track per region. Radar brightness depends on the angle the satellite looked from, so mixing ascending and descending passes makes identical ground read differently, which the model would learn as change that is not there. Among tracks covering ≥90% of the box in their worst month, the busiest wins (more passes means less speckle). The 90% rule exists because a track's strip is ~250 km wide and a box near its edge is clipped on every pass. Paris's busiest track passes 88 times a year and covers 2% of the box.

Monthly compositing. A per-pixel **median** of all passes in the month, per polarisation. The median rather than the mean because it is robust to a single bad pass. A third band records how many passes saw each pixel; a pixel with zero passes is forced to label 0 and excluded everywhere downstream, including from the area percentages. Without that, a month with partial coverage silently deflates every class and invents change at the month boundary.

Speckle filtering. There is no dedicated filter. Speckle is suppressed by two means instead: multilooking from 10 m to 50 m (25 native pixels averaged into one), and the monthly median over the month's passes (4–8 per month for most of the dataset; as few as 2.4 for some regions added in November). This is a deliberate choice, not an omission: classical speckle filters blur edges and texture, which are exactly the features a segmentation model needs, and the literature does not show a clear gain from despeckling before CNN segmentation.

Decibel conversion. `10 · log10(x)`, with the input clipped at 1e-6 first, because a zero sneaks through as −inf and then becomes NaN in every later sum.

Normalisation. Fixed ranges, not per-image statistics: VV −25…+5 dB, VH −32…0 dB, VV−VH 0…15 dB, each scaled to 0–1 and clipped. Fixed on purpose: scaling each image by its own min and max would make every image's brightest pixel read 1.0, destroying the fact that water is *absolutely* dark and cities *absolutely* bright everywhere, and it would mean the live prototype preprocessed differently from training, which is the kind of mistake that fails silently. This was verified rather than assumed: across the dataset only one region clips, namib, where 7.6% of VV pixels fall below −25 dB; it still scores 84.8% bare IoU on validation.

Channels. Three: VV, VH, and VV−VH. In decibels the difference *is* the ratio, so the third channel is the cross-polarisation ratio that separates rough-and-bright from rough-and-volumetric surfaces.

Patching. 128 × 128 tiles, stride 128 (no overlap), one tile per region-month. A tile is kept only if at least 10% of its pixels carry a label. At 50 m a tile covers 6.4 km.

Label matching. Dynamic World is 10 m; the radar grid is 50 m. Each export is laid onto an exact 5×-finer copy of the radar grid, and every 50 m cell takes a **majority vote of its 25 sub-pixels**, requiring at least half of them to be usable and a Dynamic World confidence of at least 0.35. Mixed cells such as field edges are left unlabelled rather than guessed. Every mask is then checked to sit on exactly the radar grid. A mask on a different grid describes the wrong ground, and nothing downstream would ever notice.

One label correction is applied: **cropland must green up.** Dynamic World labelled the Mongolian steppe 46% cropland at a yearly peak greenness of 0.23, where real cropland peaks at 0.7–0.8. Anything called cropland that never greens past 0.40 NDVI is relabelled grass, or bare. This is the only place optical data influences a label beyond Dynamic World's own output, and it is applied uniformly.

Augmentation. Horizontal and vertical flips, quarter-turns, multiplicative speckle (×N(1, 0.05)) and an additive calibration shift (+N(0, 0.02)), then clipped to 0–1. All radar-appropriate; no colour jitter, no shear, nothing that would imply an optical image.

Splits. By whole region, as described in §1.2.

---

2. Model

2.1 Architectures

Three networks, **all written from scratch in PyTorch**, none imported with pretrained weights:

	PARAMETERS	SHAPE
U-Net	1,942,730	4 down-blocks, base width 16, skip connections
DeepLabV3+	2,349,394	ResNet-style stem, ASPP (rates 2/4/6 at 8×8), decoder joined with layer1
SegFormer	**2,280,458**	MiT-B0-shaped: dims 32/64/128/192, depths 2/2/2/2, heads 1/2/4/6, attention reductions 8/4/2/1, all-MLP decoder 128

Input `(B, 3, 128, 128)`, output `(B, 10, 128, 128)`, one score per class including class 0, so the output channel indices line up with the class ids.

Why from scratch rather than a pretrained ResNet U-Net. Three reasons, in order of weight. **Size**: a pretrained ResNet-34 U-Net is ~24 M parameters against a dataset of a dataset that at design time was expected to be roughly 2,000 tiles from 168 ground positions and finished at 7,164 tiles from 24. Overcapacity on either is memorisation, not learning. **Domain**: ImageNet is photographs; whether edge and texture filters learned from natural images transfer to backscatter is a genuine open question, and so belongs in the experiments, not in the design assumptions. **Freedom of input**: pretrained weights force exactly three input channels, and building our own meant the input could later become nine channels for the multi-temporal experiment, which is precisely what ended up mattering most.

The honest cost of this choice is that we own the training loop's bugs. The mitigation is a discipline rather than a library: before any real run, deliberately overfit ten tiles from ten different regions. A correct model drives the loss to near zero on ten examples in minutes; if it cannot, the fault is in the code, and that takes two minutes to find instead of a day.

2.2 Training setup

Identical for all three architectures, so a difference between them is the architecture and nothing else:

epochs	40, **all of them** - no early stopping; the best epoch is kept
optimiser	AdamW, lr 3e-4, weight decay 1e-4
batch	16
schedule	2 warm-up epochs, then cosine to 1% of the learning rate at epoch 40
weight averaging	EMA, decay 0.999 (~1,000 steps ≈ 3 epochs); the average is what is scored and saved
gradient clipping	1.0
loss	cross-entropy, capped inverse-frequency class weights (cap 20×), `ignore_index=0`
selection	best validation mIoU, scored on the averaged weights every epoch

This recipe is itself a result, discussed in §5.2. The first comparison used early stopping and produced validation scores that swung 6–10 points between epochs and a "best epoch" that was partly luck.

3. Experimental results

3.1 Main results on the test set, 31-region dataset

SegFormer is the model the prototype serves, chosen on validation.

MODEL	OA	MIOU (9)	MIOU (6 MAIN)	MEAN F1 (9)	MEAN F1 (6 MAIN)
SegFormer	**66.7%**	**31.1%**	**46.6%**	**40.7%**	**61.0%**
SegFormer, 27.4 M	68.1%	32.7%	49.0%	42.2%	63.2%
SegFormer, 60 epochs	67.9%	31.5%	47.3%	41.0%	61.5%
U-Net	65.0%	31.1%	46.6%	39.6%	59.3%
DeepLabV3+	64.1%	30.3%	45.2%	39.7%	59.0%

"6 main" is the mean over water, built-up, trees, cropland, grass and bare, the six classes with measurable support. It is reported alongside, never instead of, the nine-class figure.

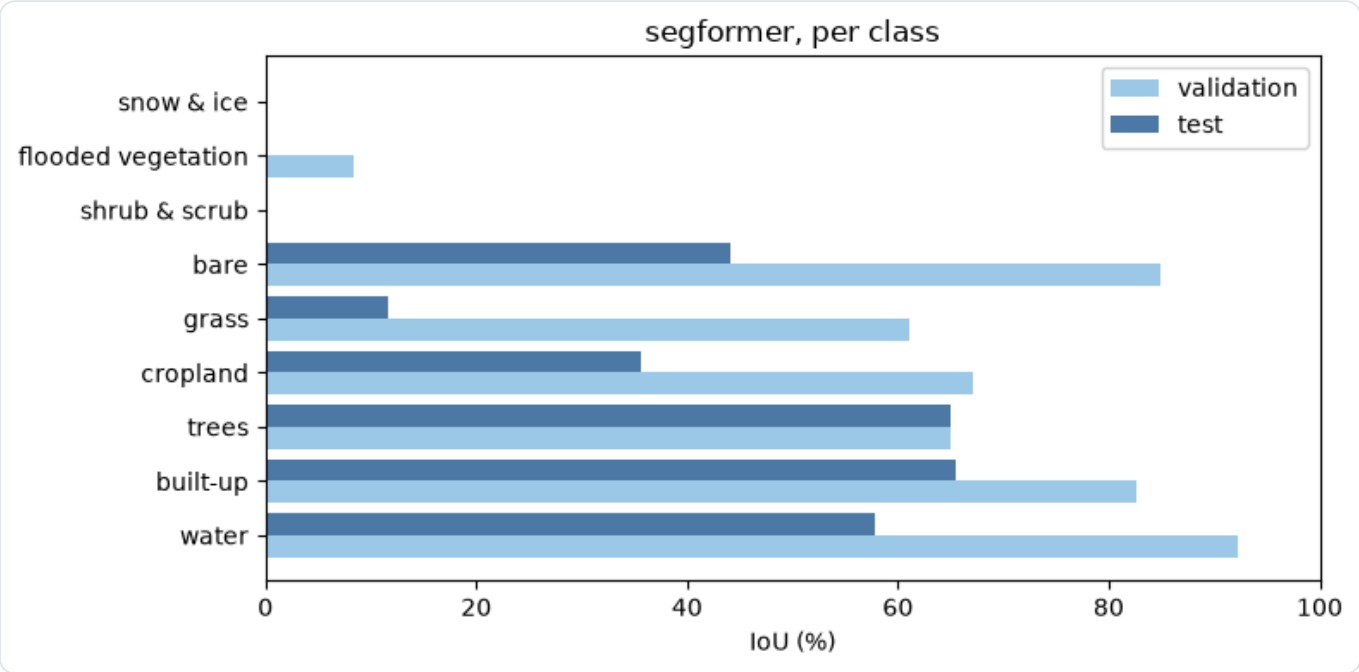


Figure 2. Per-class IoU for SegFormer. The three rightmost classes are the ones validation cannot measure.

Per-class on the test set (SegFormer):

CLASS	IOU	F1	TEST PIXELS
water	57.9%	73.3%	2.9 M
built-up	65.4%	79.1%	6.5 M
trees	65.0%	78.8%	8.7 M
cropland	35.6%	52.5%	4.7 M
grass	**11.7%**	21.0%	1.2 M
bare	44.2%	61.3%	4.8 M
shrub & scrub	0.03%	0.1%	84,700
flooded vegetation	0.0%	0.0%	16,811
snow & ice	0.0%	0.0%	26,076

F1 is always well above IoU. That is arithmetic (F1 = 2·IoU/(1+IoU)), not a better model.

Validation (SegFormer): OA 84.6%, mIoU 51.3% (9), 75.5% (6 main). Validation OA for the others: U-Net 74.7%, DeepLabV3+ 82.1%, SegFormer-60 84.8%, SegFormer-27.4 M 84.0%. The gap between 75.5% validation and

46.6% test on the same six classes is the honest measure of how much harder new ground is, and it is discussed in §5.

3.2 Training curves

RUN	EPOCHS	TRAINING LOSS	VALIDATION PEAK	AT END
U-Net	40	2.115 → 0.436	42.1% at epoch 27	38.9%
DeepLabV3+	40	1.739 → 0.370	49.0% at epoch 32	48.4%
SegFormer	40	1.755 → 0.372	51.3% at epoch 40	51.3%
SegFormer, 60 ep	60	1.814 → 0.311	51.1% at epoch 55	50.0%
SegFormer, 27.4 M	60	1.464 → 0.266	51.1% at epoch 27	50.5%

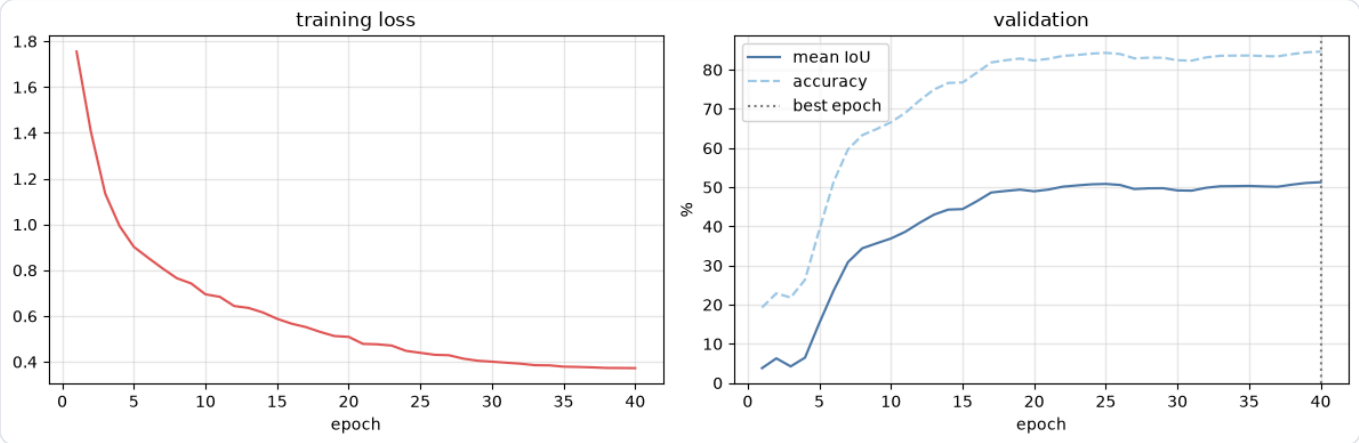


Figure 3. SegFormer, 40 epochs. Training loss falls smoothly throughout while validation flattens from about epoch 22. That gap is the model memorising its 18 training regions.

The curves are in `notebooks/01_train_unet`, `02_train_deeplab`, `03_train_seg`, `05_segformer_60_epochs` and `06_segformer_big`, each committed with its outputs.

The shape is the same in all five and it is the central diagnostic of this project: **training loss falls smoothly throughout while validation flattens.** U-Net peaks at epoch 27 and then slides back; DeepLab peaks at 32; SegFormer is the only one whose best epoch is its last. The model is memorising the training regions. More epochs or more parameters make that worse rather than better, which §3.3 confirms by measurement.

The first few epochs of every run score near zero. That is not a bug: the EMA still holds the random starting weights, and forgets them within about three epochs.

3.3 Model variants: what was tested, and what it bought

On the 31-region dataset:

QUESTION	ANSWER	EVIDENCE
Which architecture?	SegFormer, by 2.3 points on validation; on test all three tie	§3.1
Does training longer help?	**No.** 60 epochs: 51.1% val (−0.2), 31.5% test (+0.4)	notebook 05
Does a bigger model help?	**No on validation.** 27.4 M parameters (12×): 51.1% val (−0.2), 32.7% test (+1.6)	notebook 06

The bigger model's +1.6 on test comes almost entirely from two classes, bare (+10.0) and built-up (+5.8), and test bare is mostly one region, Death Valley. That is one region moving, not a broad gain. It also peaked at epoch 27 of 60: more capacity reached the same ceiling sooner, the usual sign that the data, not the model, sets it.

Two kinds of run appear in this report and they are easy to mix up. **U-Net, DeepLabV3+ and SegFormer are three different network architectures**, compared against each other on the same data with the same recipe. **Everything in the sweep below is the same SegFormer**, and only one thing changes in each row:

RUN	ARCHITECTURE	WHAT IS DIFFERENT ABOUT IT
baseline	SegFormer	nothing. The reference the others are compared against
loss: CE + Dice	SegFormer	the loss function, not the network or the data
loss: CE + Lovász	SegFormer	the loss function, chosen because it optimises IoU directly
loss: focal	SegFormer	the loss function, weighted towards pixels it keeps getting wrong
rare-class oversampling	SegFormer	how often tiles holding rare classes are drawn, not the loss
temporal_t3	SegFormer	the **input**: 9 channels, the month before, the month, the month after
temporal_mstat	SegFormer	the **input**: 9 channels, the month plus the year's mean and standard deviation

So `temporal_mstat` is not a new model. It is the same 2.28 M-parameter SegFormer shown more of the data it already had access to, which is why its result says something about the problem rather than about network design.

On the 37-region dataset (notebook 08, 40 epochs each, SegFormer throughout):

RUN	INPUT	VAL (9)	VAL (6)	TEST (9)	TEST (6)
temporal_mstat	month + year mean & std, 9 ch	**54.3%**	**80.9%**	**43.5%**	**65.0%**
temporal_t3	3-month window, 9 ch	42.1%	62.6%	27.2%	40.8%
loss: CE + Lovász	1 month, 3 ch	39.0%	58.1%	29.5%	44.2%
loss: focal	1 month, 3 ch	38.1%	56.5%	27.5%	41.2%
rare-class oversampling	1 month, 3 ch	38.1%	56.5%	29.5%	44.2%
baseline	1 month, 3 ch	36.6%	54.3%	27.4%	41.2%
loss: CE + Dice	1 month, 3 ch	36.4%	53.9%	29.3%	44.0%

This is the main experimental result of the project. Giving the network the year's mean and standard deviation alongside the month raises test mIoU from 27.4% to **43.5%** over nine classes and from 41.2% to **65.0%** over the six main ones, on seven regions it has never seen. Every class improves:

TEST IOU	BASELINE	MSTAT
water	60.7%	89.5%
built-up	58.4%	73.8%
trees	48.0%	78.1%
cropland	36.4%	49.2%
grass	15.2%	18.5%
bare	**28.2%**	**80.6%**

Bare was the worst failure in the project, with Death Valley's rock read as buildings, and it went from 28% to 81%. The mechanism is interpretable: **rock does not change through the year and vegetation does**, and the annual standard deviation is exactly that signal.

The sharper finding is the contrast with `t3`: a **three-month window gained nothing** (27.2% vs the baseline's 27.4% on test). It is not "more months" that helps. It is the whole-year summary.

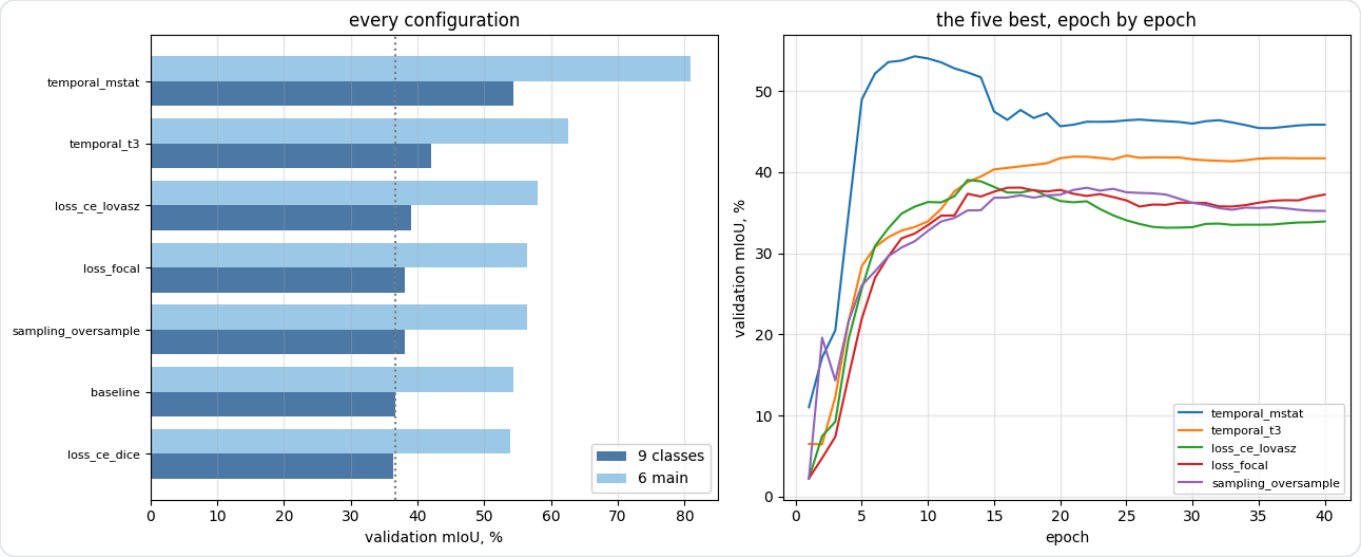


Figure 4. Every configuration in the sweep, and the five best epoch by epoch. The two bars at the top are the multi-temporal runs.

Loss functions and sampling moved results by ±2 points in both directions, which with no error bar is noise. Lovász-Softmax, which had the strongest prior reason to work because it optimises the Jaccard index directly, was the *worst* of the four on test.

3.4 Two free gains, measured without retraining

SEGFORMER-60, VALIDATION	MIOU (9)	MIOU (6)	OA
plain	51.1%	75.7%	84.8%
+ test-time augmentation (3 flips averaged)	52.0%	76.9%	85.6%
+ averaging a place's 12 months into one map	55.6%	82.6%	89.4%
+ both	**57.0%**	**84.4%**	**90.6%**

On test the same combination gives 31.5% → 33.2% (9) and 47.3% → 49.8% (6). Confirmed on four independent checkpoints. Year-averaging gives up the monthly output, so it measures the information in the sequence rather than the deliverable, and it is what pointed at the multi-temporal experiment that worked.

3.5 Six classes instead of nine

Because three classes cannot be measured on validation, we trained a model that is never asked about them: labels 7, 8 and 9 become *unlabelled*, seven output channels.

	9-CLASS MODEL, 6-MAIN COLUMN	6-CLASS MODEL	CHANGE
val, plain	75.7%	75.4%	−0.3
val, year+TTA	84.4%	82.9%	−1.5
test, plain	47.3%	49.1%	+1.8
test, year+TTA	49.8%	52.3%	+2.5

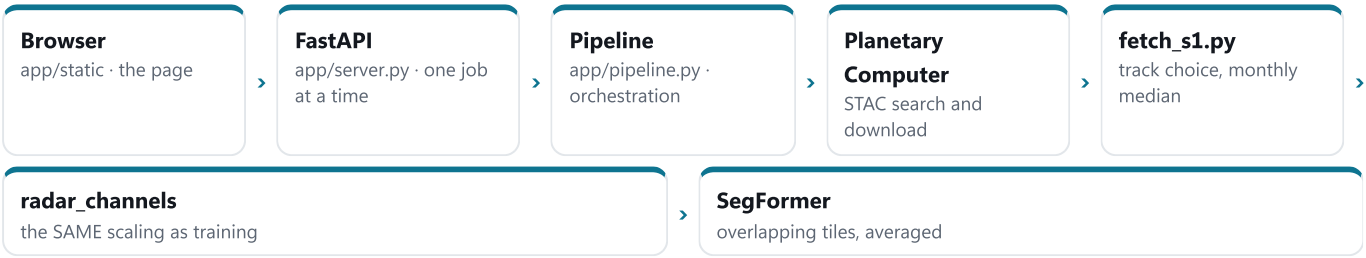
Dropping the three classes bought essentially nothing. It is slightly worse on validation, slightly better on test, and 12.7 points behind `temporal_mstat` on the same six classes. The conclusion is kept and the nine-class model

remains the deliverable: the "6 main" column was always available without removing anything, and shrub, wetland and ice are land cover that the brief asks about.

---

4. Prototype

4.1 System architecture



The single most important property of this design: **nothing in the prototype is a second copy of the preprocessing.** The app calls the same `fetch_s1` and `radar_channels` functions the training data was built with. If training and inference prepared radar differently, the model would fail silently, and no test would catch it.

The page offers a **model dropdown** listing every trained run whose weights are present, with its class count and validation score, so the models can be compared on the same place. The default is the best *validation* score among the nine-class runs. Never the best test score, because choosing on test would destroy its independence, and never across different class counts, because a six-class mIoU and a nine-class mIoU are not the same measurement.

4.2 Data retrieval and monthly compositing

1. A point, a start month and a box size (default 20 km) arrive from the page.
2. The catalogue is asked what Sentinel-1 flew over the box in those twelve months.
3. One orbit track is chosen by the same rule as the training data: among tracks covering ≥90% of the box in their worst month, the busiest.
4. Every pass is downloaded server-side cropped to the 50 m grid, four in parallel.
5. Each month becomes a per-pixel median composite with an observation-count band.
6. The twelve GeoTIFFs are read back with `radar_channels` and run through the model with tiles **overlapping by half**, probabilities averaged, so there are no tile seams.

Radar is cached per point and maps per model, so comparing two models on one place costs one model run, not another download. A new point takes a few minutes, mostly downloading; the second visit is instant.

4.3 How percentages and changes are calculated

- A pixel is counted only if the radar **saw it in all twelve months**. Every month is therefore measured on exactly the same ground, and a month with partial coverage cannot deflate every class at once.
- For each month, each class's share is its pixel count over that common area.
- **Month-to-month change** is the arithmetic difference of consecutive months' shares, in percentage points.
- **Total change** is the last month's share minus the first's, in percentage points.

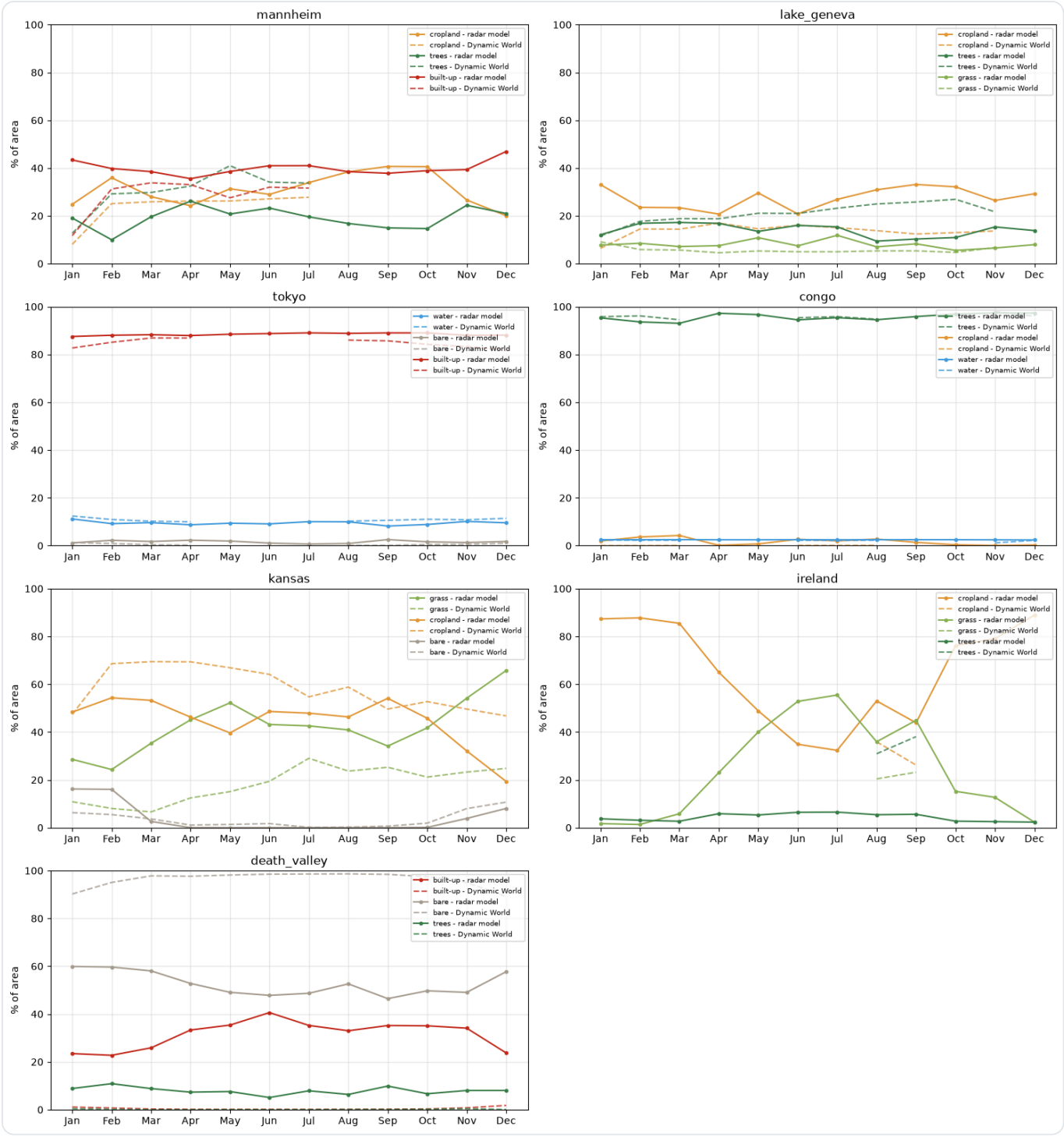


Figure 5. Class shares month by month for the test regions. Solid lines are the radar model; dashed lines are Dynamic World's own monthly shares over the same ground, exported separately from Earth Engine. Where the two move together the seasonal swing is in the ground; where they diverge it is the model.

- A thirteenth map is produced by averaging the twelve months' probabilities before the argmax, giving one yearly map that matches how the annual labels were made.

The monthly curves are also laid beside Dynamic World's own monthly shares for the same ground, exported from Earth Engine as a small CSV rather than as imagery. Dynamic World made the training labels, so this is a consistency check rather than an independent one; the independent comparison is the WorldCover one in §4.4. Months where cloud left Dynamic World seeing less than half the box are excluded, and those are exactly the months where radar is worth having.

- Each swing is compared against a **noise floor** (§5.4) and reported as no measurable change, change, or clear change.

4.4 Case study: Mannheim, 2021, 20 km box

Mannheim is a **test region**: never used in training, tuning or model selection. The radar covered 100% of the box in all twelve months, on one track.

Model output, and an independent check. ESA WorldCover 2021 (v200) is the independent map. It was used in this project only to vet regions, never as a label.

CLASS	ESA WORLDCOVER	MODEL, 12-MONTH MEAN	MODEL, YEARLY MAP	DIFFERENCE
trees	45.6%	30.2%	32.2%	**−15.4**
cropland	21.4%	27.5%	29.0%	+6.0
built-up	19.9%	36.6%	35.4%	**+16.7**
grass	11.2%	1.4%	0.5%	**−9.8**
water	1.8%	2.4%	2.2%	+0.6
bare	0.1%	0.5%	0.0%	+0.5
shrub & scrub	0.0%	1.4%	0.7%	+1.4

Water, bare and shrub land within about a point. **The model over-reports built-up by 17 points, under-reports trees by 15 and nearly cannot see grass.**

How much of that is error and how much is taxonomy is a question this comparison cannot fully separate, and it would be dishonest to present it as pure model error. Dynamic World, the label source, labels whole leafy suburbs "built", which is recorded in the build log from the labelling phase; ESA WorldCover counts the same urban trees as tree cover. A model trained on Dynamic World will therefore read a green German suburb as built-up and be "wrong" against WorldCover while being faithful to its own labels. The grass gap is different and is a real weakness: grass is the model's worst class everywhere (11.7% test IoU), and this is consistent with that.

Change over the year: cropland 20% (Dec) → 41% (Sep); trees 10% (Feb) → 26% (Apr); shrub 8–11% in winter, under 1% in summer. Against a noise floor of 7.5 points these are reported as clear change, but §5.4 explains what "change" means here.

---

5. Critical discussion

5.1 Performance on regions unlike the training data

The validation-to-test gap is the number to look at: **75.5% against 46.6%** on the same six classes, for the deliverable model. Validation is six regions each dominated by a single class: a lake, a city, a forest, a wheat plain, savanna, sand dunes. That is easy ground, and we selected models on it all project. Test is seven harder, more mixed regions.

Measured per test region, the pattern is clear:

- **Congo** (98% rainforest) and **Tokyo** (96% city) are near perfect. Uniform ground made of a class the radar reads well.
- **Kansas** is 76% cropland and gets 43–50% IoU.
- **Ireland** mixes trees, cropland and grass roughly 40/30/22 and gets 13–30% IoU each.
- **Death Valley** is 17% of all test pixels and 99% bare; only 19–49% of it is called bare. Rocky slopes and alluvial fans return radar like buildings, trees or ploughed fields. The training deserts are sand, smooth and dark. The

model learned that a desert is smooth and dark, and Death Valley is neither.

The generalisation failure is therefore not uniform: it is specific to terrain the training regions do not contain. This is a sampling problem, not an architecture problem, and 24 training regions cannot contain the world.

5.2 SAR-specific confusion between classes

52–72% of all test errors fall inside four classes: trees, cropland, grass and bare. The validation confusion matrix, read as "what percentage of this truth class went where":

TRUTH ↓	WATER	BUILT	TREES	CROP	GRASS	BARE	SHRUB	FLOOD	SNOW
water	**93.9**	1.3	0.1	0.7	2.4	0.9	0.1	0.1	0.4
built-up	0.5	**92.9**	1.9	3.2	0.3	0.1	0.9	-	0.1
trees	0.3	7.2	**68.4**	**18.7**	0.5	-	3.5	0.3	1.1
cropland	0.3	1.9	1.8	**89.3**	5.2	0.4	0.7	-	0.4
grass	0.4	0.4	2.8	**12.4**	**82.1**	0.3	1.6	-	0.2
bare	-	0.1	-	0.2	**13.7**	**86.0**	-	-	-
shrub	0.1	-	6.3	5.3	**87.4**	-	**0.8**	-	-
flooded	**36.3**	0.1	10.0	**29.1**	1.3	1.5	-	**16.9**	4.8
snow	-	-	**84.0**	16.0	-	-	-	-	-

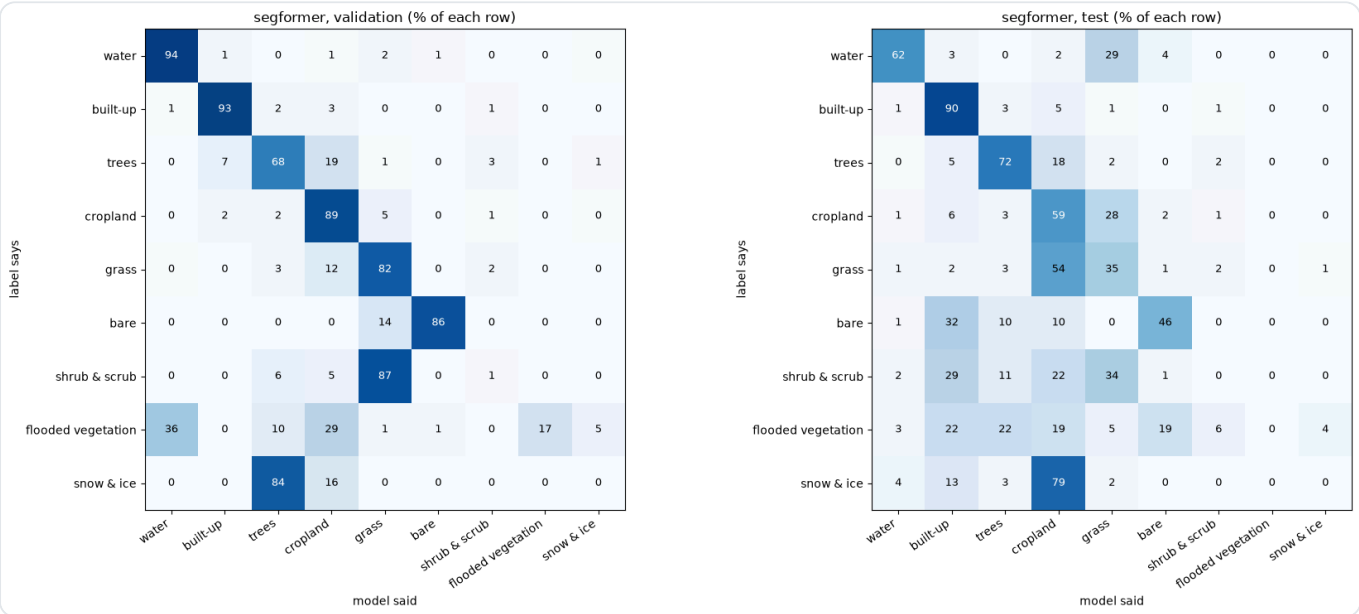


Figure 6. Validation confusion matrix, SegFormer. The bright off-diagonal cells are trees/cropland/grass and bare/grass.

The physics behind each:

- **Trees → cropland (18.7%)** and **cropland ↔ grass**. One month of radar sees roughness and wetness, nothing else. A field, a meadow and a canopy differ in how they change across the year far more than in how they scatter in any single month.
- **Bare → built-up (31.9% on test)**. Rough rock and buildings are both strong, angular scatterers. Radar has no way to distinguish a rubble slope from a housing estate.
- **Shrub → grass (87.4%)**. The model has effectively merged them. This is as much a label problem as a model one: Dynamic World's own shrub/grass boundary in savanna is not consistent.

- **Water → grass (28.7% on test)**. Wind-roughened water loses its mirror-flat signature and scatters like a vegetated surface.
- **Snow → trees (84%)**. Measured on 324 validation pixels; not a meaningful number.

The clearest single lesson in this project is that the fix for the first of these is not a better network. It is more time. `temporal_mstat` closed much of that gap by being shown the year, after a 12× larger model and 50% more epochs closed none of it.

5.3 Seasonal effects that look like land-use change

This is a structural problem, not a bug, and it deserves to be stated plainly.

The labels are annual. The inputs are monthly. The model was never shown a "winter field" label. It was shown one label per place for the whole year, and twelve months of radar all carrying that same label. It is therefore trained month-invariant and then asked for month-varying output. That tension is inherent to the task as set.

The consequences are visible and reproducible:

- Irish pasture is called **cropland in winter** (wet, bare-looking soil) and **grass in summer**. Nothing on the ground changed.
- Kansas shows a **bare** share in January–February that is most likely frozen ground: ice changes how soil reflects radar.
- Mannheim's winter "shrub" at 8–11% is bare winter fields.

So the correct reading of every number this prototype produces is **land cover as the radar sees it that month**, not a change of land use. A harvested field genuinely *looks* like grass to a radar. Reporting it as land-use change would be false; reporting it as what it is, is useful.

5.4 How classification errors inflate the change percentages

Directly, and by a large factor.

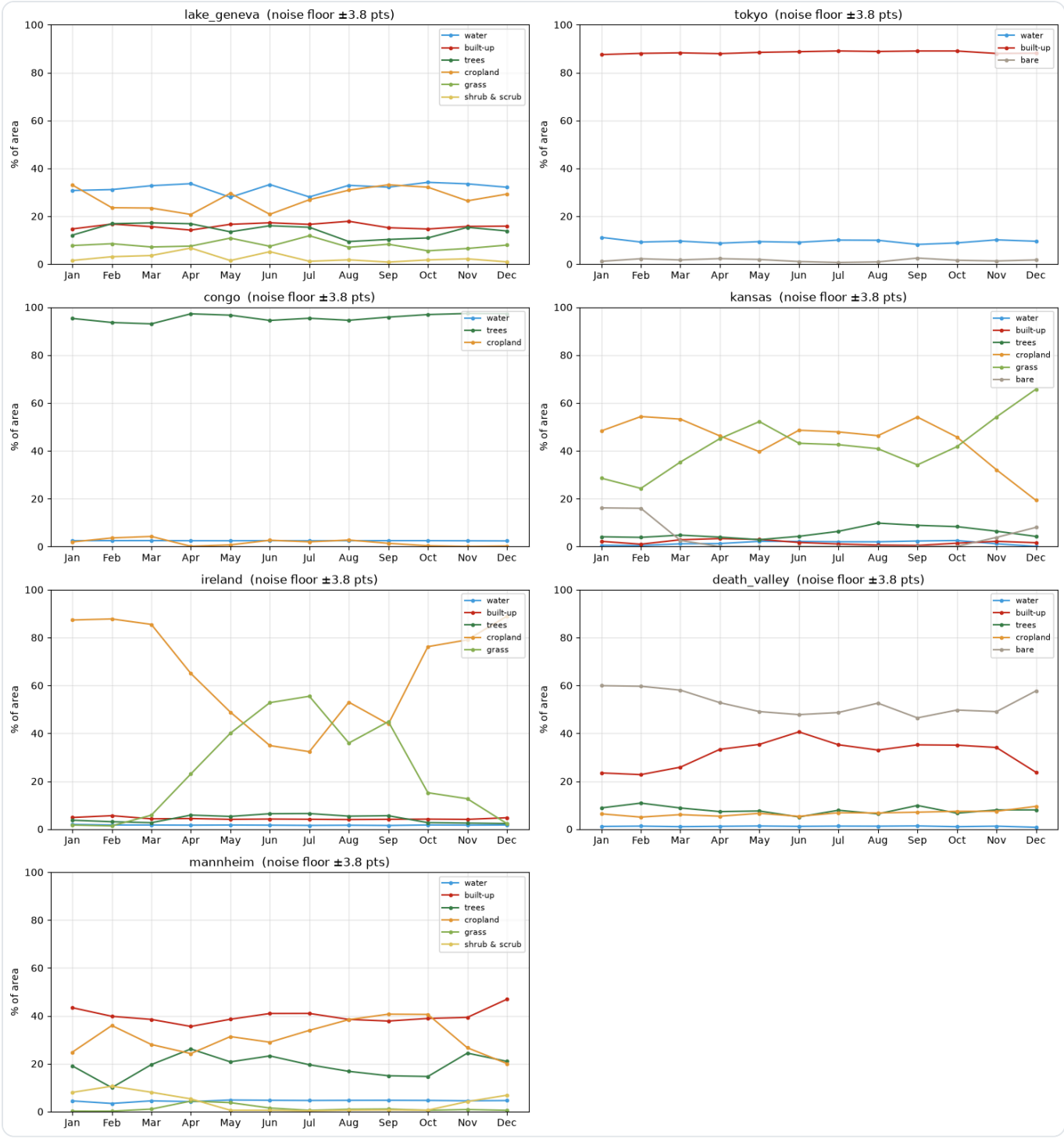


Figure 7. Monthly change against the noise floor. Bars inside the grey band are smaller than the model's own wobble and are reported as no measurable change.

A city cannot appear or vanish in a year. So the month-to-month swing of the predicted built-up share over test regions is a direct measurement of the model's own wobble. That **noise floor is 7.5 percentage points**.

That figure is itself contaminated, and the build log records why: the noise floor selects regions by *predicted* built-up share, and Death Valley's predicted built-up swings 23% → 41% across the year while its label is 0.1% built-up. A false city was used as a reference city. **Excluding Death Valley, the median swing of the real cities (Tokyo 1.5, Lake Geneva 3.7, Mannheim 11.3) is 3.7 points, not 7.5.**

Either way, the implication stands. The deliverable model's class shares swing **12–18 percentage points** between months at Mannheim. Against a floor of 3.7–7.5, a good part of that is the model changing its mind, not the ground changing. Any change smaller than the floor is reported as no measurable change, and that rule is applied in the prototype rather than left to the reader.

The multi-temporal model shows the opposite failure, and it is worth recording because it was measured, not predicted. Run on the same Mannheim radar, `temporal_mstat` changes only **2.3% of pixels** between January and July, against the deliverable model's **22.1%**. Its monthly shares move by 0.5–1.0 points. Because six of its nine input channels are identical for all twelve months, it leans on the annual summary and effectively produces one map repeated twelve times.

Accuracy and sensitivity to change pull in opposite directions here. The single-month model over-reports change; the whole-year model cannot see it. That is why the prototype ships the single-month model despite `temporal_mstat` being 24 points more accurate. The brief asks for month-to-month change, and a model that reports zero change every month answers the accuracy question while failing the task.

5.5 Limits of the evaluation itself

- **Every score is agreement with Dynamic World**, which is 73–84% accurate. We cannot exceed our labels, and part of what is counted as model error is label error.
- **Three classes are unmeasurable on validation** (0.42%, 0.08% and 324 pixels), yet each carries one ninth of the headline mean. Roughly one third of the nine-class validation number is arithmetic over classes the split cannot measure.
- **Grass test rests on ~99,000 pixels in the first dataset.** It is the thinnest class anywhere, and the least certain number in this report. Worse, the region chosen for it shows the label problem directly: ESA WorldCover calls Ireland 77% grass; Dynamic World calls it 13.7% grass, 22.6% trees and 17.5% cropland, and leaves 40% unlabelled.
- **Validation is six regions**, so each class's score is essentially one region's. A small shift in a decision boundary flips a large contiguous area, which is why early runs swung 6–10 points per epoch.
- **No cross-validation and no repeated seeds were run.** Differences under about 2–3 points anywhere in this report should not be treated as real. The cross-validation experiment is written and ready in notebook 08 but was not run for time.

5.6 What was tried and did not work

Recording these matters as much as the successes:

ATTEMPT	RESULT
Training 50% longer (60 epochs)	−0.2 validation. No gain.
A 12× larger model (27.4 M)	−0.2 validation; +1.6 test, almost all from one region.
Dice loss	+1.5 val / −0.5 test at 12 epochs; −0.2 val at 40. Noise.
Lovász-Softmax	+2.4 val / worst of four on test.
Focal loss	+1.5 val / −1.0 test. Noise.
Rare-class oversampling	+1.5 val / +2.1 test. Within noise.
Three-month input window	+5.5 val, **0.0 test**.
Dropping to six classes	−0.3 val / +1.8 test. Nothing.
Early stopping (first attempt)	Produced a 21–27 point validation-test gap and a "best epoch" that was partly luck. Abandoned.

6. Challenges, and what was done about them

Four regions came back partly empty. The orbit track was being chosen by image count alone, and a track whose 250 km strip clips the box gives many passes over a sliver of ground. Fixed by requiring ≥90% box coverage in the worst month *before* ranking by count.

Dynamic World called the Mongolian steppe 46% cropland at a peak greenness of 0.23. Fixed with a uniform rule: cropland that never greens past 0.40 NDVI is relabelled. The rule is documented rather than hidden, because it is the one place an optical signal touches a label.

Every overfit test printed FAIL and the runs went ahead anyway. The test was taking the first ten tiles alphabetically, all Amazon and all January, where two classes covered a few hundred pixels and held the averaged score near 50% however well the rest was learned. Fixed by taking one tile from each of ten different regions, and judging on pixel accuracy above 90%.

Validation was unstable and early stopping made it worse. Three causes, all fixed: runs stopped while the learning rate was still high, so every scored epoch was a large-step snapshot; the best epoch was picked off a noisy curve; and validation is structurally noisy because it is six regions. The answer was to run all 40 epochs, let the cosine fade complete, score an EMA of the weights rather than any single step, and clip gradients. Scores went *down* and became trustworthy, which was the right trade.

Restoring all nine classes dropped the headline from ~48% to ~31%, without the model changing at all. On the six classes the earlier runs had, test stayed at 45–47%. Three classes that score near zero each count as one ninth. We kept them and report both numbers.

Rare classes had no data. Flooded vegetation was 0.026% of training pixels, about one tile's worth in the entire set. Six regions were scouted against live catalogues and added, lifting the three rare classes to 6.9%, 2.0% and 8.0%. The fix worked on the training side and the validation metric still cannot see it, because validation was deliberately left untouched so the runs stayed comparable.

A notebook that never ran. `07_segformer_rare_classes` was written but never executed, and its result stayed "pending". It was removed rather than shipped empty; the question it asked was answered from the input side by notebook 08.

Two dataset versions, one repository. Re-running the training notebooks on the larger dataset produced a second, different set of results for the same model names. Both are real. The canonical files describe the weights that ship; the re-runs are kept beside them in `outputs/retrain_24regions/` rather than overwriting or being discarded.

---

7. Future work

In the order a next team should attempt them.

1. Resolve the accuracy/sensitivity trade-off (§5.4). This is the most interesting open problem the project produced. `temporal_mstat` is 24 points better and nearly blind to monthly change. A model given the month *and* a summary of the year, trained with a loss that explicitly rewards getting the month right, or with the annual channels randomly dropped during training so it cannot rely on them alone, might get both. Nothing in the literature we read addresses this directly.

2. Wire the multi-temporal model into the prototype. `app/pipeline.py` already downloads all twelve months and now computes the annual statistics; serving `mstat` is about fifteen lines plus a cache invalidation. Worth doing once (1) is settled.

3. DEM channels: elevation, slope, local incidence angle. The single clearest aim at the largest remaining error: bare → built-up at 31.9% on test is terrain geometry, not land cover. Published results on terrain correction in mountains are mixed (one Sentinel-1 study reports 13–20% gains, a later one from the same group reports that flattening *hurt*), but feeding slope as an input channel is lower-risk than reprocessing the radar.

4. More regions. The ceiling is 24 independent ground positions. Downloads are free and anonymous; the cost is the Earth Engine export clicks for labels. Fifty to sixty regions is the obvious next step and needs no new ideas.

5. Region-level cross-validation. Five folds over the 37 regions would recover the 37% of data currently spent on evaluation and, just as importantly, give an error bar, without which half the comparisons in §3.3 cannot be called. Written and ready in notebook 08.

6. In-domain self-supervised pretraining. All 37 regions × 12 months is ~440 scenes of unlabelled radar. The literature is strongest here for the few-labelled-scenes case: one reported result has a BYOL-pretrained U-Net with three labelled scenes beating a supervised model trained on seven. Expensive, and the highest-ceiling option on this list.

7. Label quality. Use Dynamic World's confidence bands as soft targets, or raise the 0.35 floor. This changes the labels, so every number would move and nothing would be comparable, so it deserves its own study rather than being slipped into a comparison.

8. The ImageNet comparison that was promised and never run. `segmentation-models-pytorch` is installed for it and notebook 08 has the experiment written. One pip install. The literature disagrees with itself about whether ImageNet transfers to backscatter, so either outcome is publishable as a finding.

Not recommended: going to 30 m. It would require re-downloading every region, re-importing every label, rebuilding every tile and retraining every model; it contradicts the documented reading of "do not train on high-resolution data"; and it would trade away the speckle suppression that multilooking to 50 m provides. None of the measured failure modes is resolution-limited.

---

8. What was delivered

Dataset	37 regions × 12 months of Sentinel-1 at 50 m, with Dynamic World labels on the identical grid
Models	U-Net, DeepLabV3+ and SegFormer, written from scratch and trained identically, plus five variants
Best deliverable model	SegFormer, 2.28 M parameters - test OA 66.7%, mIoU 46.6% over the six measurable classes, 31.1% over all nine
Best model measured	`temporal_mstat` - test mIoU 43.5% (9), 65.0% (6 main)
Prototype	`app/` - a point and a month in, twelve monthly maps, class shares and change out, with a model selector
Notebooks	eight, each committed with its outputs and figures
Build log	`docs/changes.md` - every phase, what broke, and how it was fixed

The honest summary, with each number named:

A small model trained from scratch on radar alone labels 66.7% of pixels correctly on seven regions it has never seen, and reaches a mean IoU of 46.6% over the six classes the evaluation split can actually measure. The one change that moved any of this was showing the model the whole year instead of a single month, which took that figure to 65.0% and the nine-class mean to 43.5%. The nine-class mean for the

prototype's model is 31.1%, and that lower number is mostly the cost of keeping three classes we could not measure rather than quietly dropping them.